

P3: Control Flow

Name: Alston Liow Guan You

Student ID: 37440985

Date: 03 August 2026

Overview

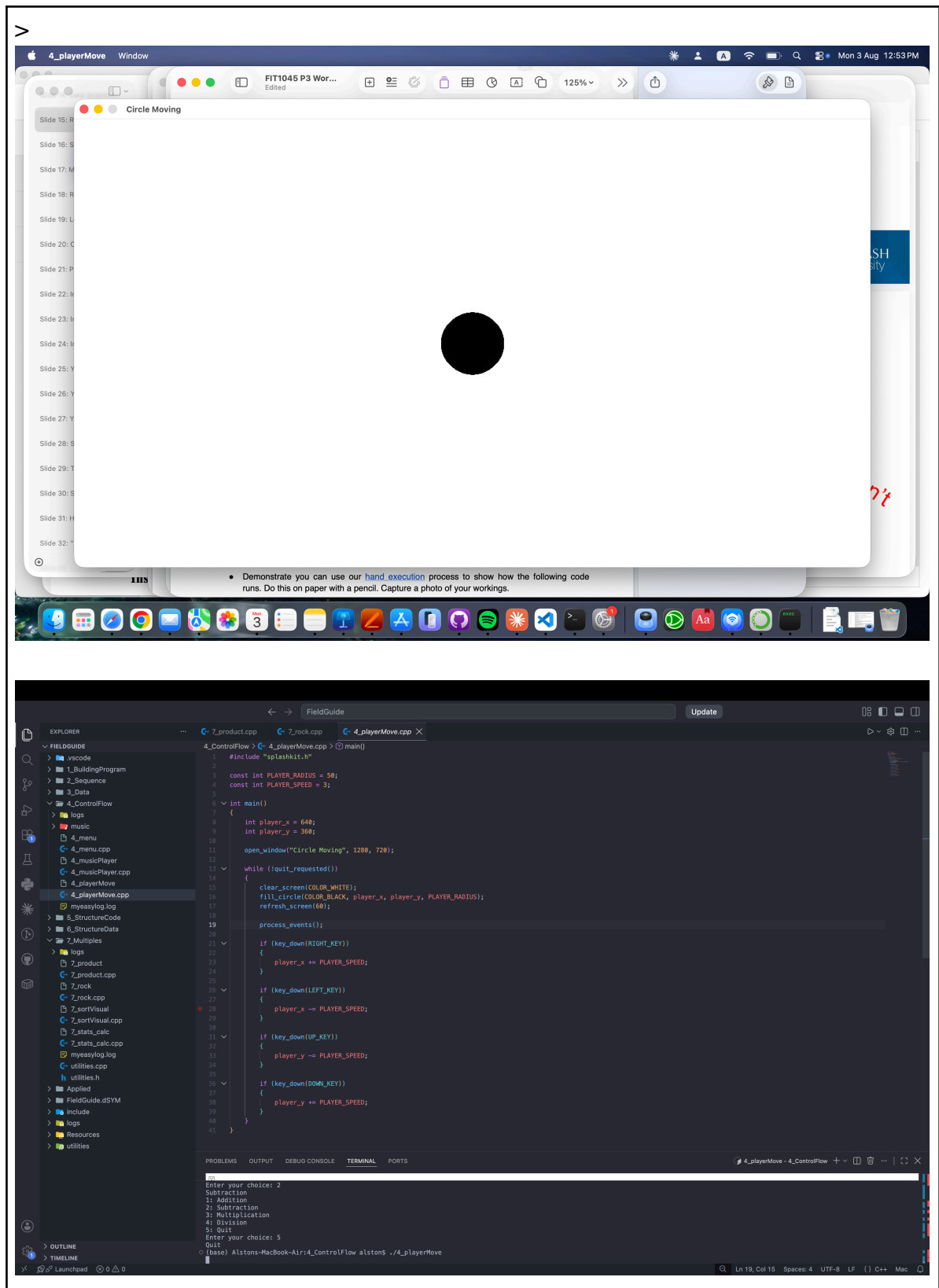
Learn to use control flow statements to take your programs to the next level, making them interactive and responsive. This will build upon your understanding of sequence and how the computer executes instructions, showing how you can manipulate this sequence using control flow statements.

Tip: More complex task

This is a big and important topic! You will need to dedicate some time to this, and you need to make sure that you really understand these ideas as they are central to everything in programming. Ensure you make use of all the support available to help you learn to use these new tools effectively.

Do the following:

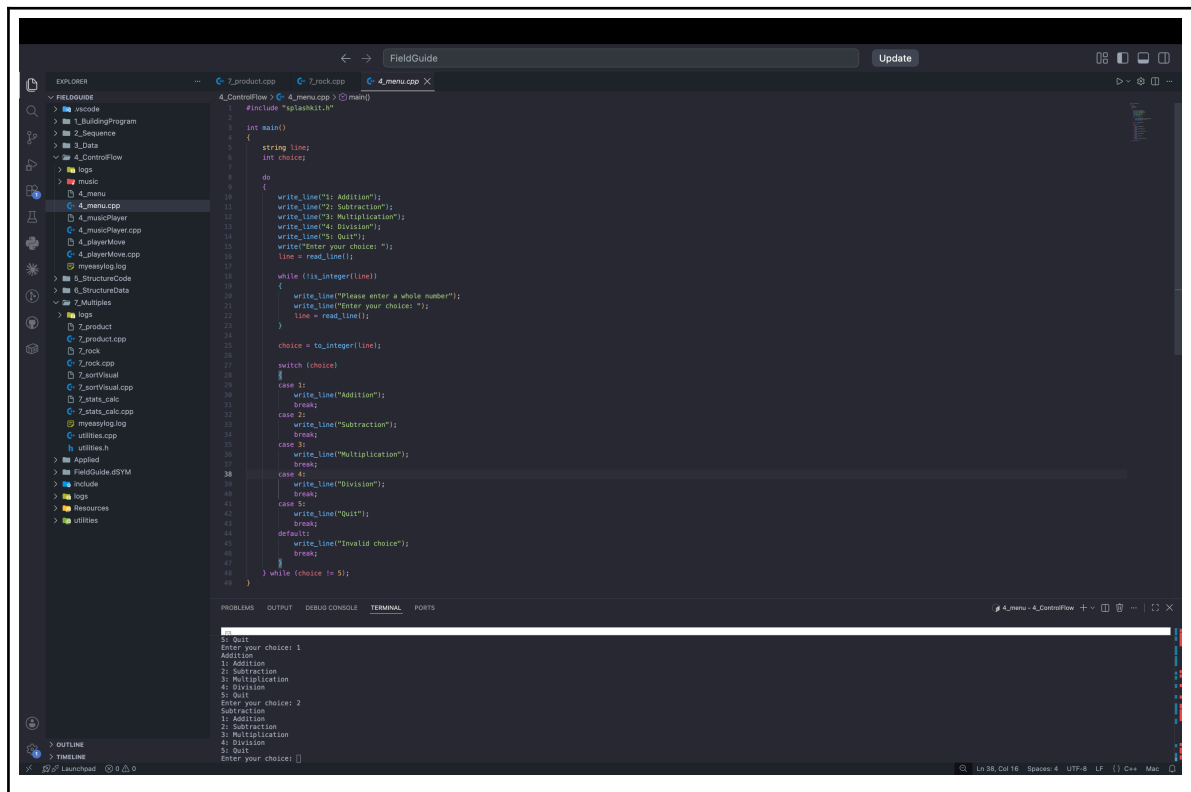
- Everything you need is in the [Control Flow](#) chapter of our [Programmer's Field Guide](#). The [Intro Tours](#) walk through the concepts, and should get you started.
- Add a screenshot showing you were able to create an interactive GUI program. You can use the [player move tour](#), which goes across a number of pages.



Tip: Remember, you have agency

Remember that you are responsible for your learning, and we are happy to support your agency in relation to this. If you do not want to follow our guided tours, you can propose alternate tasks. We just ask that you discuss this with the teaching team first.

- Add a screenshot showing you were able to create a terminal based program driven by a menu. You can use the [menu and input validation tour](#).



- Demonstrate you can use our [hand execution](#) process to show how the following code runs. Do this on paper with a pencil. Capture a photo of your workings.

```
int target;
int i = 1;
string stars = "";

write("Count to: ");
target = to_integer(read_line()); // User enters 3

while (i <= target)
{
    stars = stars + "*";
    write_line(stars);
    i++;
}
```

Notes _____ Date: _____

Memory

target 3

i ~~1~~ 2 3 4

star ~~" "~~ ~~" "~~ ~~" "~~ "****"

Output Terminal

Count to :

★

★★

★★★

>

- Repeat for the following code:

```

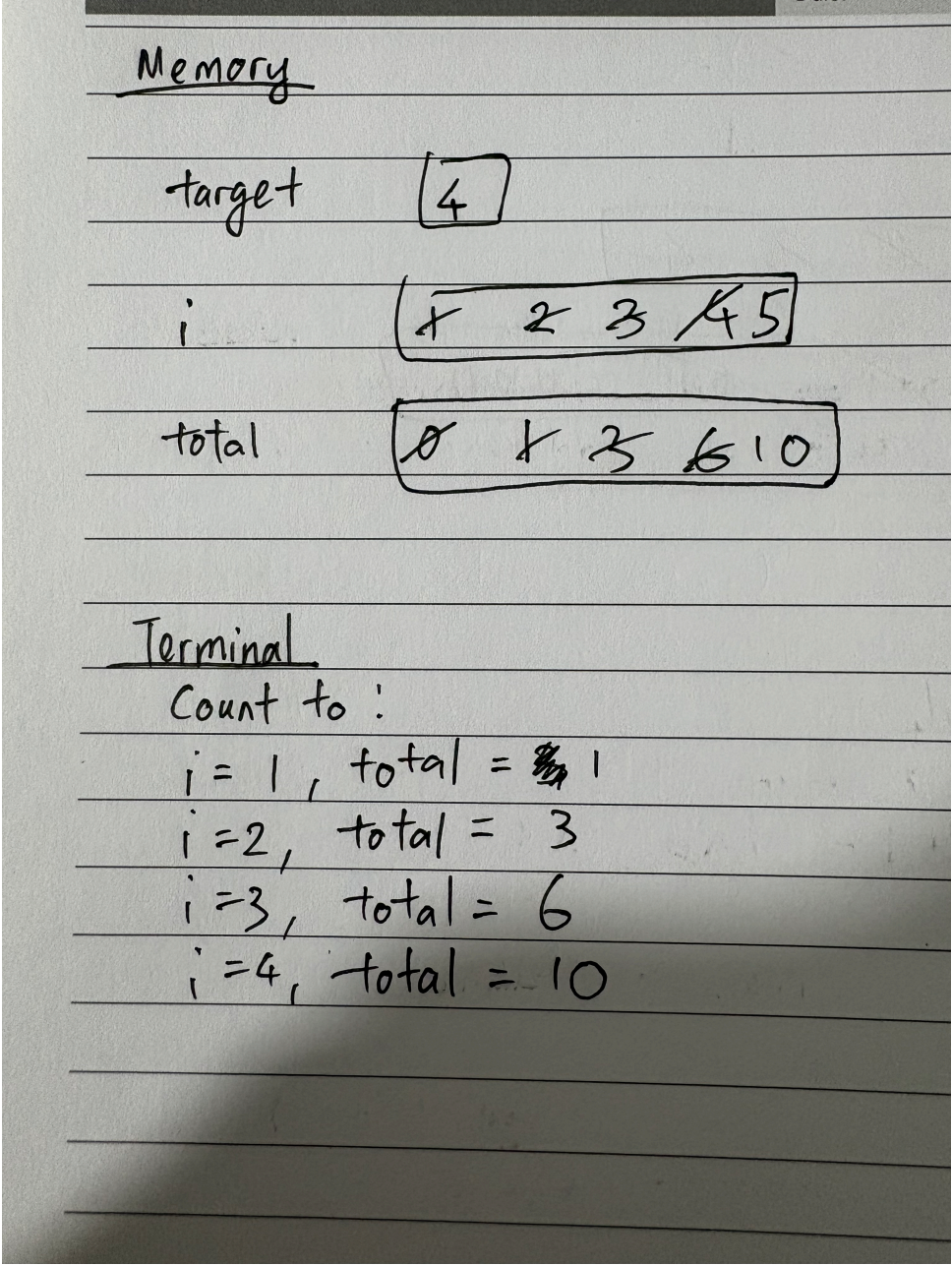
int target;
int i = 1;
int total = 0;

write("Count to: ");
target = to_integer(read_line()); // User enters 4

while (i <= target)
{
    total = total + i;
}

```

```
write("i = ");  
write(i);  
write(", total = ");  
write_line(total);  
  
i++;  
}
```



Memory

target 4

i ~~1~~ 2 3 ~~4~~ 5

total ~~0~~ 1 3 ~~6~~ 10

Terminal

Count to :

i = 1, total = ~~1~~ 1

i = 2, total = 3

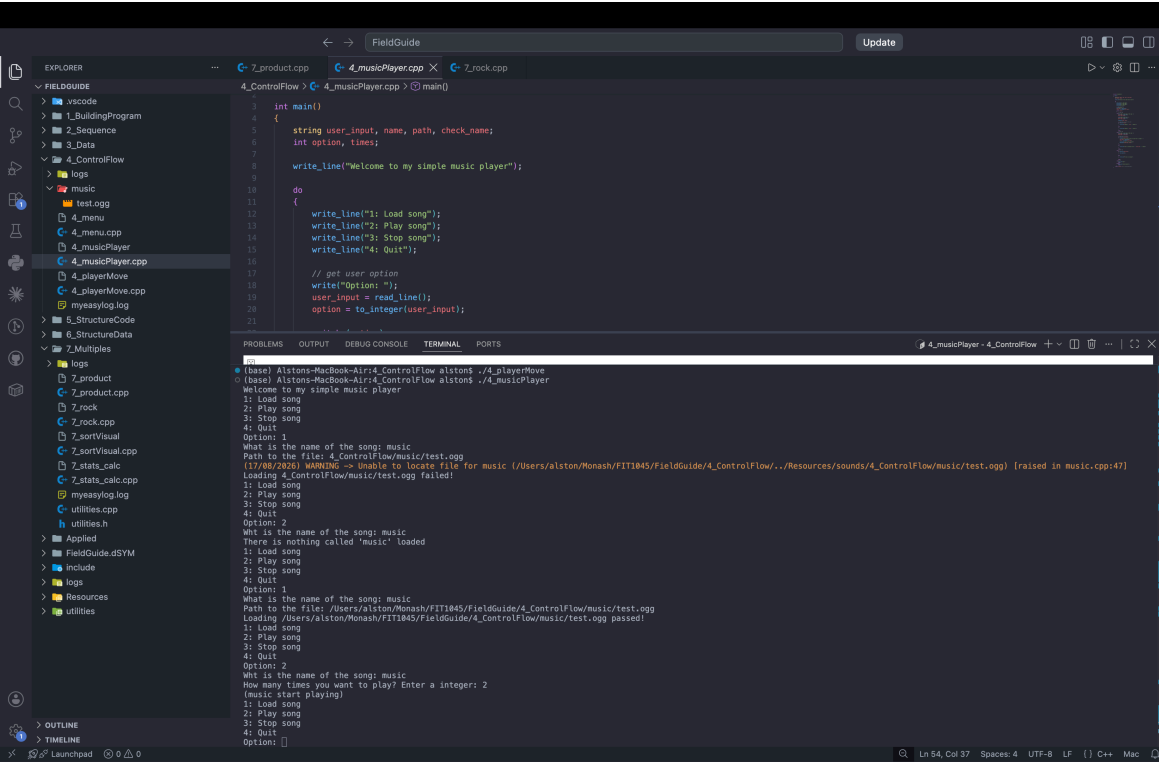
i = 3, total = 6

i = 4, total = 10

>

- Complete one of the [Test Your Capabilities](#) tasks from the chapter to see that you can now put these tools together to achieve something yourself.

> This be more complex when compare to P2, I think I need to repeat this many time. I also write every thing need to output first, and the go to switch to complete the function. In 1, 3, 3 need to use has_music() to prevent bug. In case 1, I load music using variable name "name", in case 2 I use a new variable "check_name" , so that user can load various song, in case 3, I also use has_music() to check the validity so then pause it. But when I think clear about this project structure, just need spend time to code and debug.



The top screenshot shows the IDE with the following components:

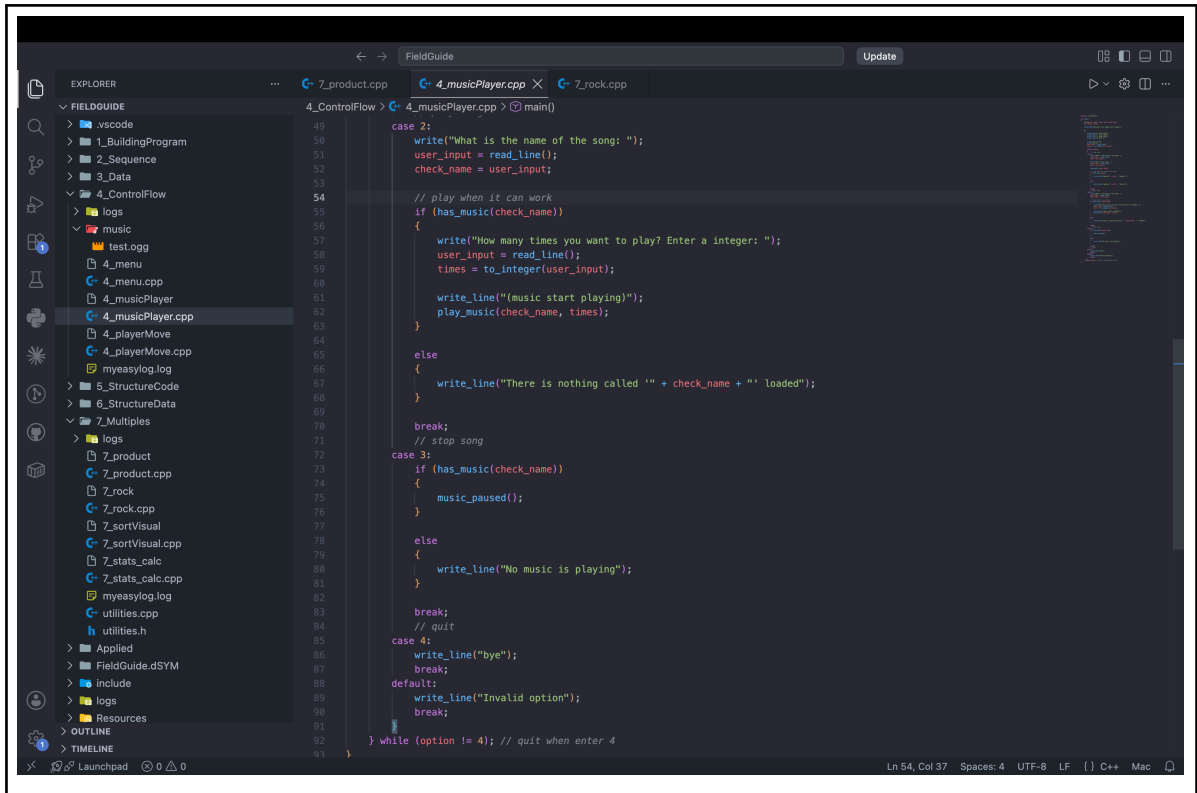
- Explorer:** Lists project files including `4_musicPlayer.cpp`, `4_playerMove.cpp`, `myeasylog.log`, `5_StructureCode`, `6_StructureData`, `7_Multiples`, `logs`, `product`, `rock`, `sortVisual`, `stats`, `stats_calc`, `utilities`, `utilities.h`, `FieldGuide.dSYM`, `include`, `resources`, and `utilities`.
- FieldGuide:** Shows the execution flow of the program, including the `main` function and the `do` loop.
- Problems/Output:** Displays error messages and program output. The output shows the program running successfully, with the user inputting "music" and "2".

The bottom screenshot shows the source code for `4_musicPlayer.cpp`:

```

1 #include "splashkit.h"
2
3 int main()
4 {
5     string user_input, name, path, check_name;
6     int option, times;
7
8     write_line("Welcome to my simple music player");
9
10    do
11    {
12        write_line("1: Load song");
13        write_line("2: Play song");
14        write_line("3: Stop song");
15        write_line("4: Quit");
16
17        // get user option
18        write("Option: ");
19        user_input = read_line();
20        option = to_integer(user_input);
21
22        switch (option)
23        {
24            // Load song
25            case 1:
26                write("What is the name of the song: ");
27                user_input = read_line();
28                name = user_input;
29
30                write("Path to the file: ");
31                user_input = read_line();
32                path = user_input;
33
34                load_music(name, path);
35
36                // load when can success the path
37                if (has_music(name))
38                {
39                    write_line("Loading " + path + " passed!");
40                }
41                else
42                {
43                    write_line("Loading " + path + " failed!");
44                }
45            }
46        }
47    }
48 }

```



```

49
50
51     write("What is the name of the song: ");
52     user_input = read_line();
53     check_name = user_input;
54
55     // play when it can work
56     if (has_music(check_name))
57     {
58         write("How many times you want to play? Enter a integer: ");
59         user_input = read_line();
60         times = to_integer(user_input);
61
62         write_line("music start playing");
63         play_music(check_name, times);
64     }
65
66     else
67     {
68         write_line("There is nothing called '" + check_name + "' loaded");
69     }
70
71     break;
72     // stop song
73 case 3:
74     if (has_music(check_name))
75     {
76         music_paused();
77     }
78
79     else
80     {
81         write_line("No music is playing");
82     }
83
84     break;
85     // quit
86 case 4:
87     write_line("bye");
88     break;
89 default:
90     write_line("Invalid option");
91     break;
92 } while (option != 4); // quit when enter 4
93

```

Tip: Coding style and language use

Make sure to follow our [style guide](#). All of the code in our Field Guide follows this style, so you'll have lots of examples to work from. OnTrack will check this when you submit.

Also ensure that you are only using concepts covered so far. We want you to focus on these concepts and how they work together.

Reflect:

- What was the most interesting thing you learnt through this task, and why?

> I learn about switch(){}, which can clearly structure my program, and also can let user make a choice toward any the function written by me. This make my program more interesting, useful and can do more thing.

- How do you feel sequence and control flow are connected? Does understanding sequence help you understand control flow?

> Exactly yes, sequence is very important, for example in control flow we use do, ifelse all follow sequence.

- Looking back, how long did it take to complete this task? What did you spend the most time on?

> Maybe 4 hours, I spend most time in my music player, need some time to think about how to build it and debug.

Tip: Weekly study expectations

Remember units are designed to take around 144 hours of study to pass.

- University study requires you to take responsibility for your time, which takes some getting used to. You need to put in an appropriate amount of time, as well as make sure that your time is used effectively. How are you going with managing your study time?

> I like study at night or weekend, I want to spend maybe 10 hours in 2-3 days for every two task and field guide per week, and maybe 2 hours for online c++ learning without splashkit.

Disclose:

- Explain your use of Generative AI for this task. If you didn't use it, just write "I did not use any AI for this task". If you did use it, please state which AI model you used (e.g. ChatGPT) and how.

> I use claude to explain some splashkit syntax and function. All code is written by me.

Submit:

- Export this worksheet as a PDF.
- Upload your submission to OnTrack:
- Engage with feedback and discussion to get this signed off.
 - To get the task **Complete**, you need to engage with the feedback you receive. Your tutor will review your submission and may ask you to clarify aspects of the task or include aspects you have missed.
 - Remember, a **Resubmit** status is not a bad thing – your tutor is just trying to help improve your understanding. When you do resubmit, make sure to include a comment that describes what you changed. This needs to demonstrate how you have addressed all of your tutor's feedback.